

Marcio Juany Zaballa
22200024

20/06/2026

I=

1- La SSOT actúa como la única fuente confiable de datos dentro de la aplicación.

Su objetivo es abstraer el origen de los datos para que la UI no necesite saber si vienen de

Base de Datos Local o de una API Remota

Siendo que el flujo normalmente funciona así...

1ero La UI solicita datos al View Model

2do El View Model llama al Repository

3ero El Repository decide:

Obtener datos desde la Base de Datos si ya existen o Consultar la API con Retrofit si necesitan datos nuevos.

4to Si llegan datos de la API, se guardan en ROOM

5to La UI obtiene al ROOM y se actualiza automáticamente

Entonces la UI depende del Repository y no del origen real de los datos.

2- ¿Qué es el View Model?

Es un scope de características asociada al ciclo de vida del View Model.

Permite ejecutar lo que son tareas mínimas para mantener el hilo principal.

¿Porque es importante para la memoria?

Porque Esto: Memory leaks, características huérfanas y Operaciones camino invencionalmente.

¿Qué sucede si el usuario cierra la pantalla?

Si una petición de red fue iniciada dentro de View Model y el usuario llega a salir de la pantalla.

1. El View Model Se destruye

2. El View Model se cancela

3. Todas características hijas se cancelan.

Esto ocurre gracias a Structured Concurrency, donde las tareas hijas dependen del ciclo de vida del scope padre.

3. Difference entre Flow & Cold State plan

Flow

Vs

Kold Stateflow

- Comienza a emitir datos cuando diges la calote
 - Normalmente sale
 - ideal para stream topología
- Es totalmente independiente de si diges lo observa.
 - Mantiene el ultimo valor emitido
 - ideal para manejar ~~esta~~ estado de UI

Clara se prepara el state flow para AI?

Porque la interfaz nunca realmente conoce el estado actual [Error, loading, success]

- Si la pantalla rota o se reacciona, presionar el botón central inmediatamente
- No conectar después una nueva emisora

Esto ha sido ~~pero~~ ~~agradable~~ ~~reacción~~ como
Anchaid:

9.- La DI manual consiste en crear e integrar dependencias manualmente, sin framework como Maven o Dagger. En el lado opuesto lo hacemos desde la clase Application.

¿Para qué sirve hoy?

Permite que un objeto se cree solo cuando se necesita por primera vez.

Reduce la carga de desarrollo, menor costo de implementación. Evita crear objetos innecesarios al arrancar la app.

La Dependencia que trae de DI Manual

Bajo cumplimiento, Mayor flexibilidad.

Facilita el Testing y una mejor organización del código.

Basado en DI manual permite controlar la creación de dependencias con mayor precisión, incluso como la base de datos.

II-

1- El flujo de datos se serializa automáticamente cada vez que una tabla en ROOM cambia su estado usando el tipo Byte

Objeto: flow

2- El operador parallel permite ejecutar tareas pesadas fuera del Main Thread, aprovechando optimizada para tareas CPU como el procesamiento de datos.

Objeto: Default

3- Para ser un servicio en primer plano de una aplicación se debe declarar en el manifest con el tipo android:foregroundServiceType="background"

Objeto: foregroundServiceType="background"

4- La función startService() debe darse aplicación en la que garantiza que los componentes ejecutan la misma instancia del componente mediante un cont

Objeto: onCreate()

5.- El componente que permite transformar una API basada en callbacks antiguo en una faceta surgida moderna de libra

Resp: Async and Cancellable Coroutine

6.- El archivo de ~~metadatos~~ metadatos que describe la estructura de los datos del sistema operativo se llama extensión

Resp: AndroidManifest.xml

7.- En la arquitectura Android la capa que actúa como punto estándar entre el framework y los controladores se denomina HAL

Resp: HAL

8.- El operador de Flavor que permite combinar múltiples funciones (como GPS y Sensor) para crear un entorno completo de funciones

Resp: combine

III :-

1.-

class VersionRepository { private var versionDao : VersionDao }

fun getVersion() = versionDao.getVersion()

}

on Application :

class Application : Application() {

val database by lazy {

AppDatabase.getInstance(this)

}

val repository by lazy {

VersionRepository(database.getVersionDao())

}

}

BOOK

2=

a) Crear data class Usuario
data class Usuario {

Val nombre : String

Val apellido : String

Val edad : Int?

b) Crear un objeto con los datos reales

Val usuario = Usuario {

nombre = "Hernandez"

apellido = "Juan Zeballos"

edad = 23

c) Calcular el Puntaje de Identidad

Val puntajeIdentidad : Int = usuario.

nombre.replace(" ", "").length

usuario.apellido.replace(" ", "").length

d) Imprimir el ~~total~~ promedio

```
println("Usuarios: $usuarios, nombre: $  
$ usuarios, apellido: $, tu promedio de identidad  
es: $ promedio identidad")
```

e)

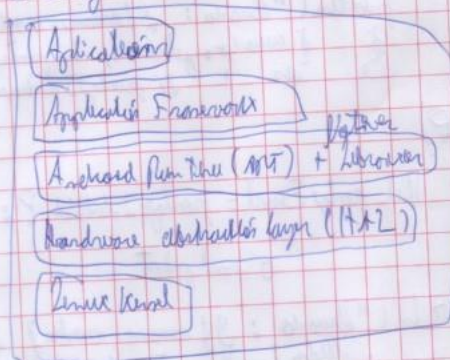
Val promy Final = promedio identidad ? : 0

o eval print

```
printLn("Usuarios: $usuarios, nombre: $usuarios  
apellido: $, tu promedio de identidad es:  
$ $ promedio identidad ? : 0 $")
```

IV:

1. Gràfics



2.

Aplicacions → Donen suport a les aplicacions de l'usuari.

Aplicatiu Framework → Proporciona APIs i serveis per desenvolupar apps Android.

ART → És el motor d'execució de les aplicacions Android.

Natius Linux → Biblioteca escrita en C/C++ per donar suport a les aplicacions.

HAL → Interfície de programari entre el hardware i el sistema operatiu.
Linux kernel → És el sistema operatiu de baix nivell.

